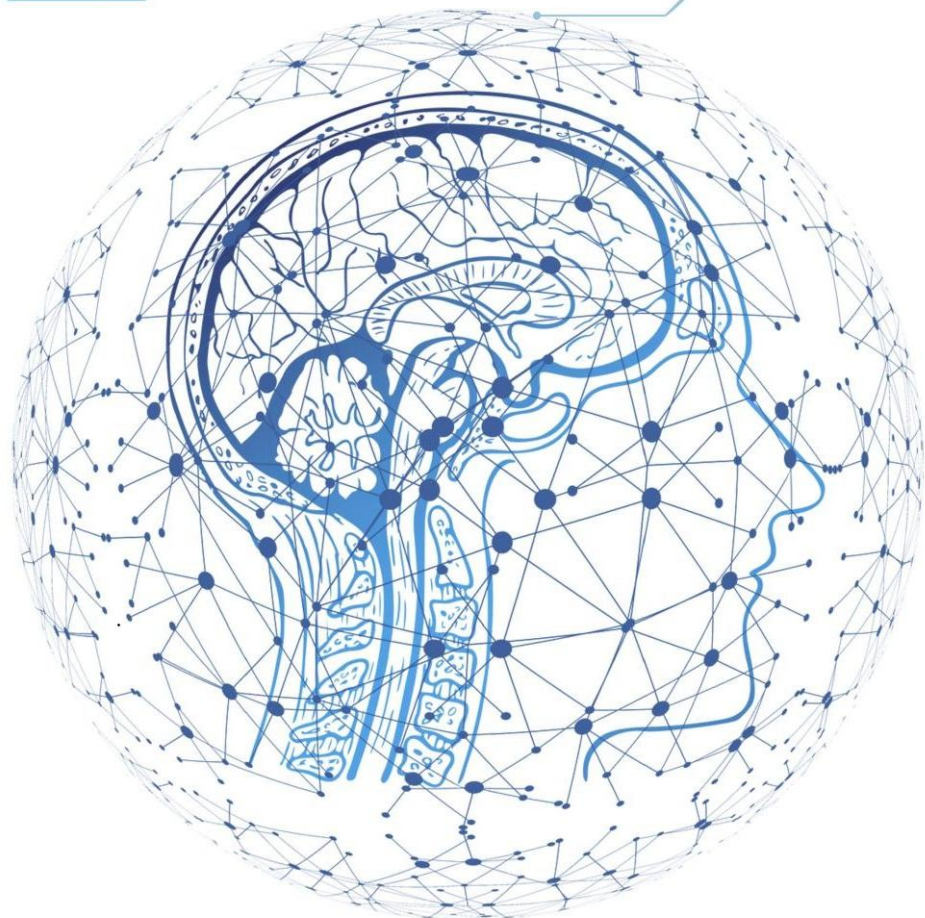


DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

www.hoasen.edu.vn

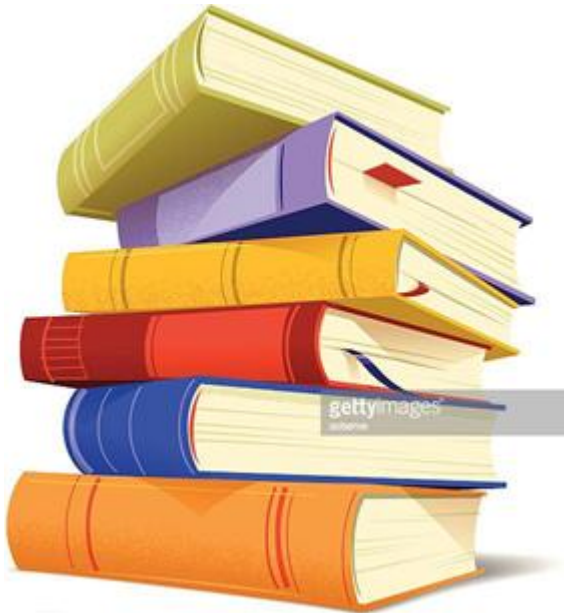
Chương 9



Advanced deep learning
for computer vision
Học sâu nâng cao
cho thị giác máy tính

Contents

Nội dung

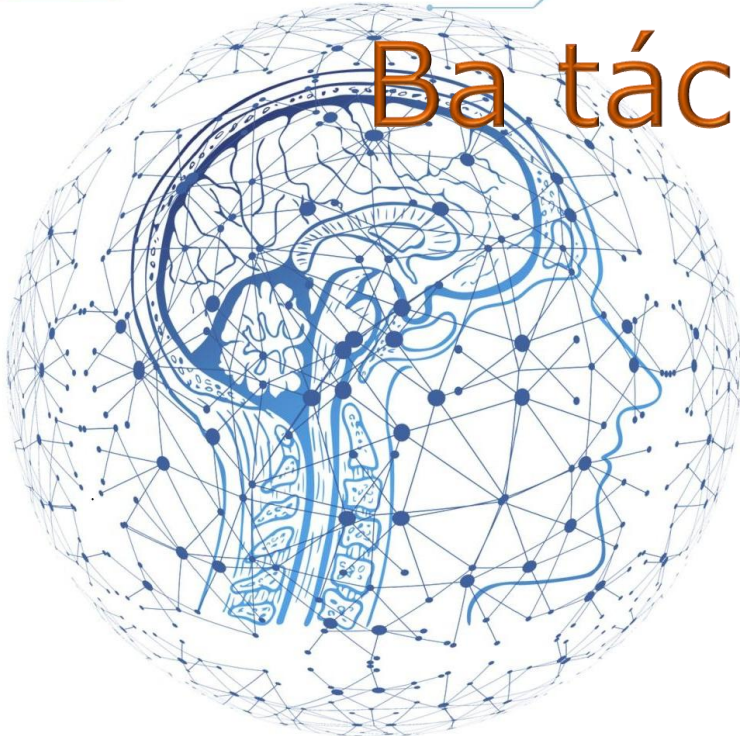


- Ba tác vụ thị giác máy tính cốt lõi
- Ví dụ phân đoạn ảnh
- Các mẫu kiến trúc convnet hiện đại
- Diễn giải những gì convnet học được

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Three essential computer vision tasks
Ba tác vụ thị giác máy tính cốt lõi



Computer vision goes beyond classification

Thị giác máy tính không chỉ là phân loại

- Chương 8 tập trung vào **image classification**: gán nhãn cho toàn ảnh.
- Chương 9 mở rộng sang các tác vụ thị giác máy tính **giàu cấu trúc hơn**.
- Ba tác vụ nền tảng: **classification, segmentation, object detection**.
- Điểm chung: mô hình phải học biểu diễn không gian từ pixel, cạnh, texture đến đối tượng.
- Các kỹ thuật kiến trúc và diễn giải trong chương này dùng được cho nhiều bài toán ảnh.

Three essential tasks Ba tác vụ cốt lõi

- **Image classification:** gán một hoặc nhiều nhãn cho toàn bộ ảnh.
- **Image segmentation:** gán nhãn cho từng pixel hoặc từng vùng trong ảnh.
- **Object detection:** xác định các đối tượng bằng **bounding box** và nhãn lớp.
- Đầu ra càng chi tiết thì mô hình càng phải giữ thông tin **vị trí không gian** tốt hơn.
- Chương này đi sâu vào **segmentation**, kiến trúc convnet hiện đại và interpretability.

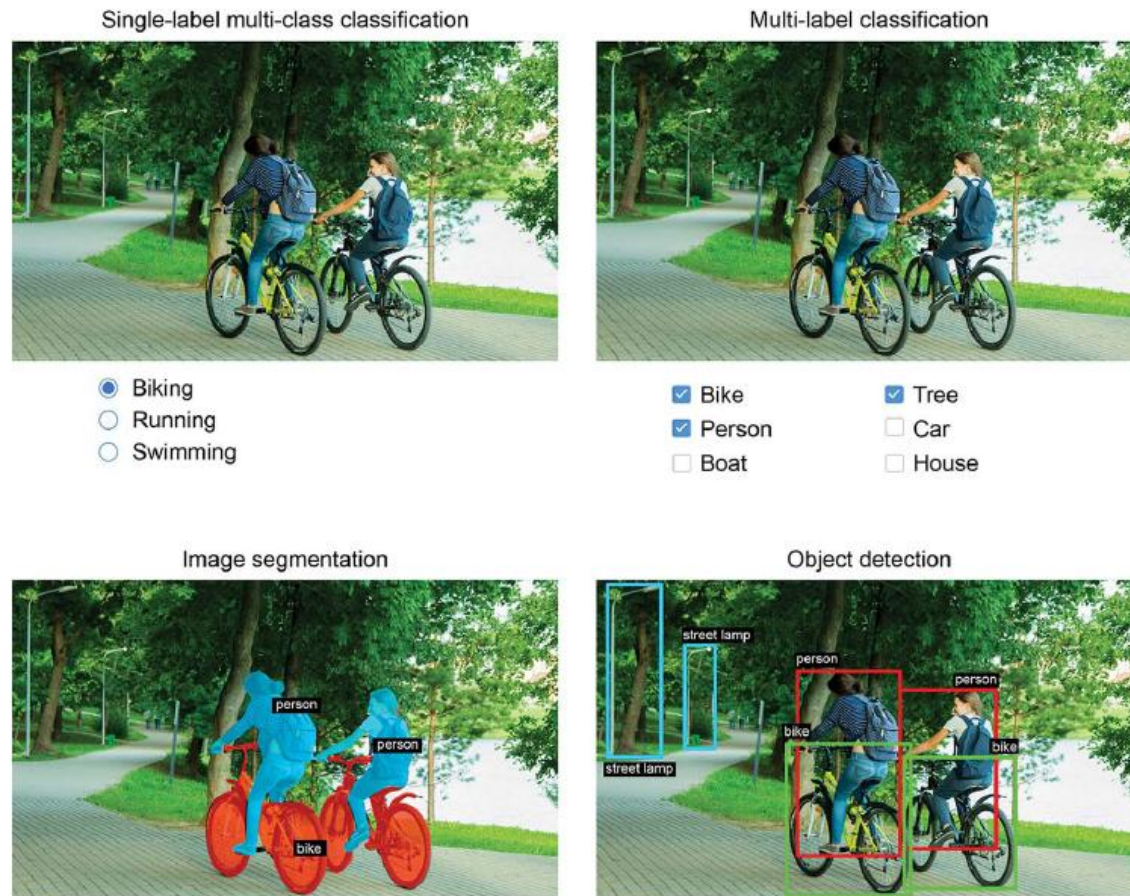


Figure 9.1 The three main computer vision tasks: classification, segmentation, detection

Hình 9.1. Phân loại ảnh, phân đoạn ảnh và phát hiện đối tượng.

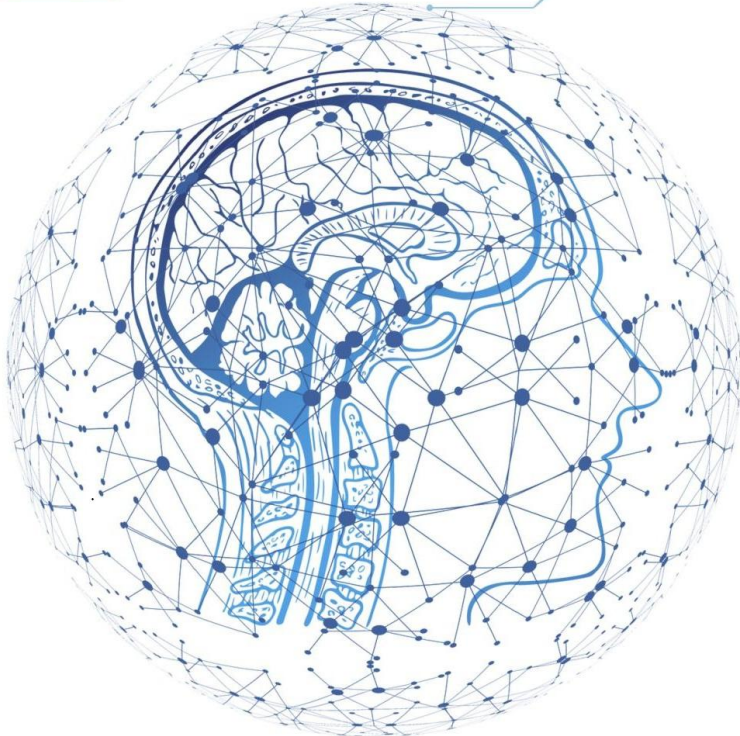
Many other vision tasks Nhiều tác vụ thị giác khác

- Ngoài ba tác vụ chính còn có **image similarity, keypoint detection, pose estimation**.
- Có thể dự đoán **3D mesh**, nhận diện khuôn mặt, hoặc theo dõi đối tượng qua video.
- Sách chỉ giới thiệu object detection ở mức khái niệm, không triển khai chi tiết.
- Lý do: detection thường cần pipeline phức tạp hơn, ví dụ RetinaNet, YOLO hoặc Faster R-CNN.
- Nền tảng để học tiếp vẫn là: convolution, representation, architecture và interpretability.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

An image segmentation example
Ví dụ phân đoạn ảnh



Semantic vs. instance segmentation

Phân đoạn ngữ nghĩa và phân đoạn thể hiện

- **Semantic segmentation:** mọi pixel cùng class được gán cùng một nhãn.
- **Instance segmentation:** tách riêng từng đối tượng cụ thể trong cùng class.
- Ví dụ: hai con chó trong ảnh có cùng class 'dog' nhưng là **hai instance** khác nhau.
- Semantic segmentation phù hợp khi cần bản đồ lớp; instance segmentation phù hợp khi cần đếm vật thể.
- Trong sách, ví dụ Oxford-IIIT Pets là **semantic segmentation**.



Figure 9.2 Semantic segmentation vs. instance segmentation

Hình 9.2. Phân đoạn ngữ nghĩa so với phân đoạn thể hiện.

Oxford-IIIT Pets dataset

Bộ dữ liệu Oxford-IIIT Pets

- Bộ dữ liệu có **7,390 ảnh** chó và mèo thuộc nhiều giống khác nhau.
- Mỗi ảnh có một **segmentation mask** đi kèm.
- Mask là ảnh một kênh, cùng kích thước với input, chứa **nhãn nguyên cho từng pixel**.
- Trong sách: nhãn gồm **foreground, background** và **contour**.
- Sau khi xử lý, các nhãn được đưa về dạng **0, 1, 2** để train với 3 class.



Figure 9.3 An example image

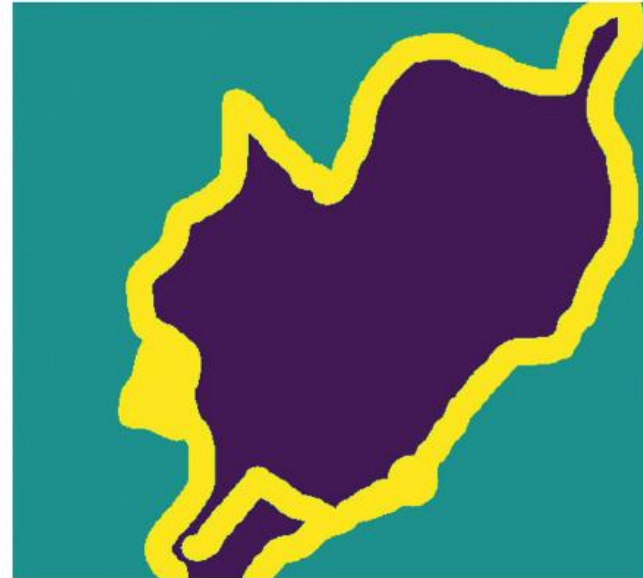


Figure 9.4 The corresponding target mask

Hình 9.3-9.4. Ảnh đầu vào và segmentation mask tương ứng.

Preparing segmentation data Chuẩn bị dữ liệu phân đoạn

- Sắp xếp danh sách **image paths** và **mask paths** theo cùng thứ tự.
- Shuffle hai danh sách bằng **cùng seed** để ảnh và mask luôn khớp nhau.
- Resize input và mask về **200 x 200** để batch có kích thước cố định.
- Input ảnh dùng shape `(200, 200, 3)`; mask dùng shape `(200, 200, 1)`.
- Tách **1,000 mẫu validation** để theo dõi overfitting.

Encoder-decoder segmentation model

Mô hình phân đoạn dạng encoder-decoder

- **Encoder:** dùng `Conv2D` với `strides=2` để giảm kích thước không gian.
- Số filter tăng dần: **64 -> 128 -> 256**, feature map nhỏ hơn nhưng giàu thông tin hơn.
- **Decoder:** dùng `Conv2DTranspose` để phóng to feature map về kích thước ban đầu.
- Layer cuối `Conv2D(3, activation="softmax")` dự đoán **3 class cho mỗi pixel**.
- Output có dạng `(200, 200, 3)` : tại mỗi pixel là phân phối xác suất trên 3 lớp.

Downsampling without losing location Giảm mẫu nhưng vẫn giữ vị trí

- Sách dùng **strided convolution** thay vì `MaxPooling2D` trong encoder.
- Lý do: segmentation rất nhạy với **vị trí pixel**.
- `MaxPooling2D` có thể làm mất chi tiết vị trí trong cửa sổ pooling.
- `Conv2DTranspose` học cách **upsampling** thay vì chỉ phóng to cố định.
- Mục tiêu là cân bằng giữa **ngữ cảnh rộng** và **độ chính xác không gian**.

Training and prediction

Huấn luyện và dự đoán mask

- Loss dùng trong sách: **sparse categorical crossentropy**.
- Vì mask lưu class id nguyên nên không cần one-hot encode từng pixel.
- Dùng **ModelCheckpoint** để lưu model có validation loss tốt nhất.
- Overfitting xuất hiện khoảng **epoch 25** trong ví dụ của sách.
- Khi dự đoán, dùng `argmax` trên trục class để lấy nhãn cuối cùng cho mỗi pixel.

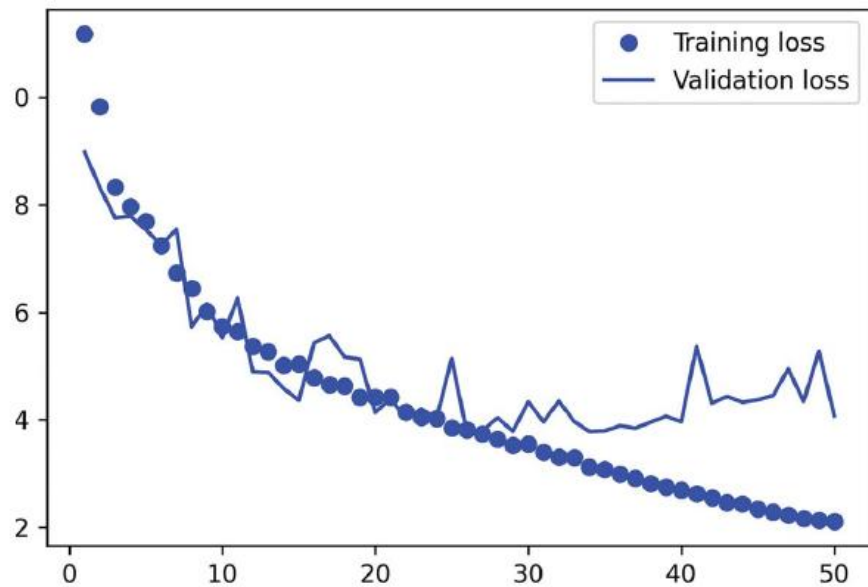


Figure 9.5 Displaying training and validation loss curves



Figure 9.6 A test image and its predicted segmentation mask

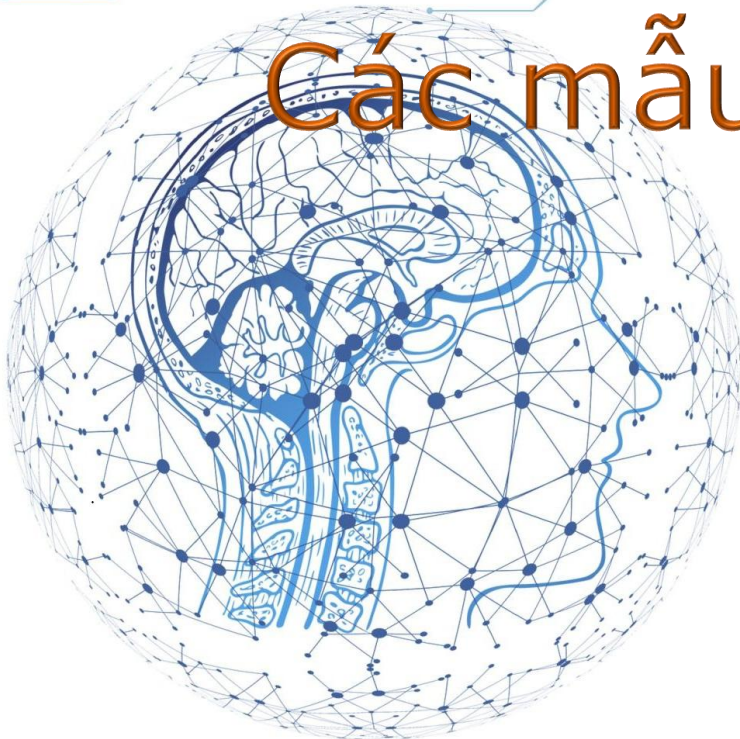
Hình 9.5-9.6. Loss huấn luyện/validation và mask dự đoán.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Modern convnet architecture patterns

Các mẫu kiến trúc convnet hiện đại



What architecture means

Kiến trúc mô hình nghĩa là gì

- **Architecture** là cách chọn layer, tham số layer và cách nối các layer.
- Kiến trúc xác định **hypothesis space**: tập các hàm mà model có thể học.
- Kiến trúc tốt mã hóa **prior knowledge** về bài toán.
- Nó giúp gradient descent tìm nghiệm tốt dễ hơn, không chỉ tăng số layer tùy ý.
- Trong computer vision, prior quan trọng là tính cục bộ, chia sẻ trọng số và cấu trúc phân cấp.

Modularity, hierarchy, reuse

Tính mô-đun, phân cấp và tái sử dụng

- **Modularity:** chia hệ thống phức tạp thành các block nhỏ.
- **Hierarchy:** tổ chức block theo tầng, từ đặc trưng đơn giản đến trừu tượng.
- **Reuse:** dùng lại cùng loại block hoặc phép convolution ở nhiều vị trí.
- Nhiều convnet mạnh dùng các block lặp lại thay vì từng layer rời rạc.
- **Ablation study:** bỏ từng thành phần để kiểm tra thành phần nào thật sự tạo ra cải thiện.

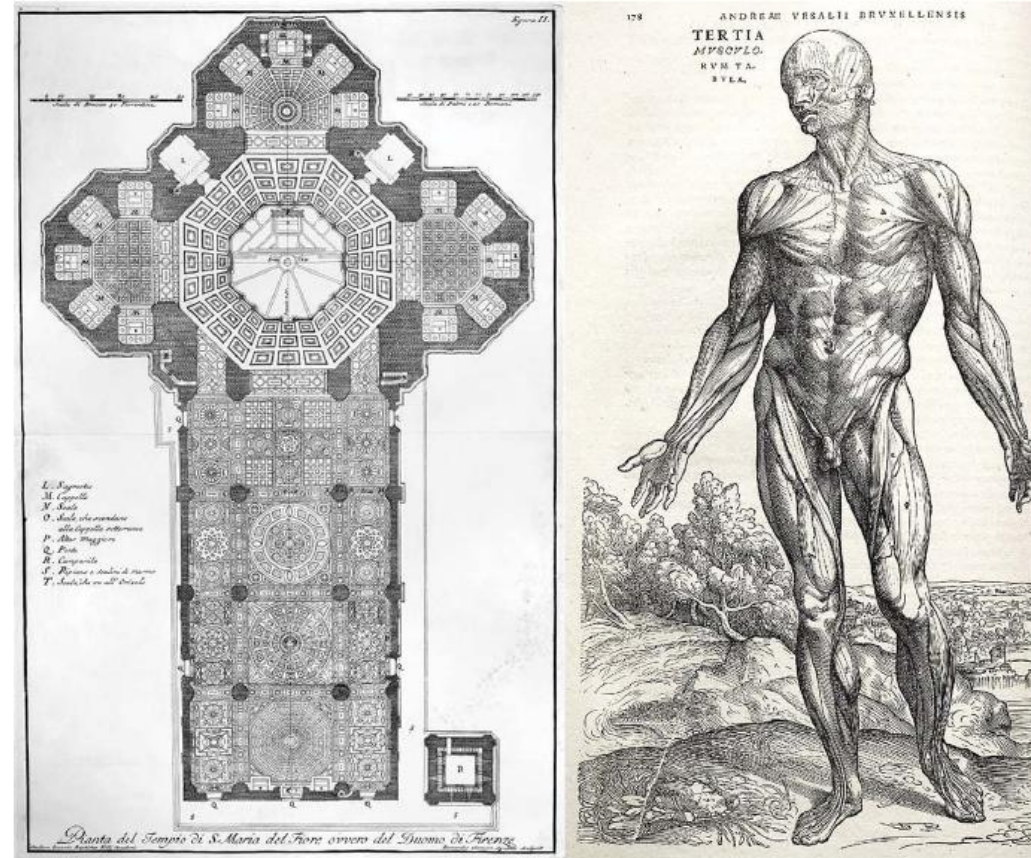


Figure 9.7 Complex systems follow a hierarchical structure and are organized into distinct modules, which are reused multiple times (such as your four limbs, which are all variants of the same blueprint, or your 20 “fingers”).

Hình 9.7. Hệ thống phức tạp thường có cấu trúc phân cấp và mô-đun.

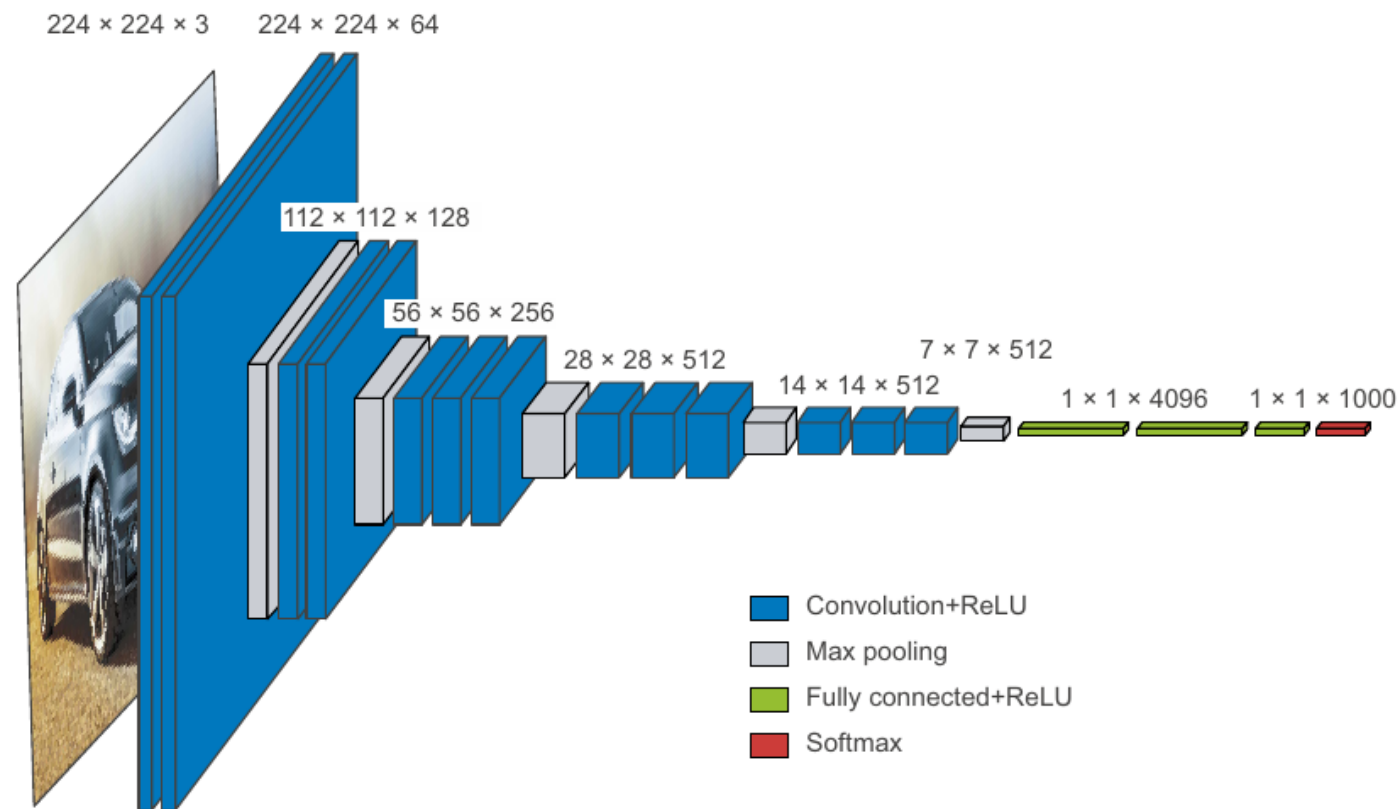


Figure 9.8 The VGG16 architecture: note the repeated layer blocks and the pyramid-like structure of the feature maps

Hình 9.8. Kiến trúc VGG16 với block lặp lại và cấu trúc hình tháp.

Residual connections

Kết nối dư

- Mạng rất sâu dễ gặp vấn đề **vanishing gradients**.
- Residual connection thêm input của block vào output của block.
- Công thức: $y = F(x) + x$
- Đường tắt giúp thông tin và gradient đi qua mạng dễ hơn.
- Ý tưởng này là nền tảng của họ **ResNet** và nhiều kiến trúc hiện đại.

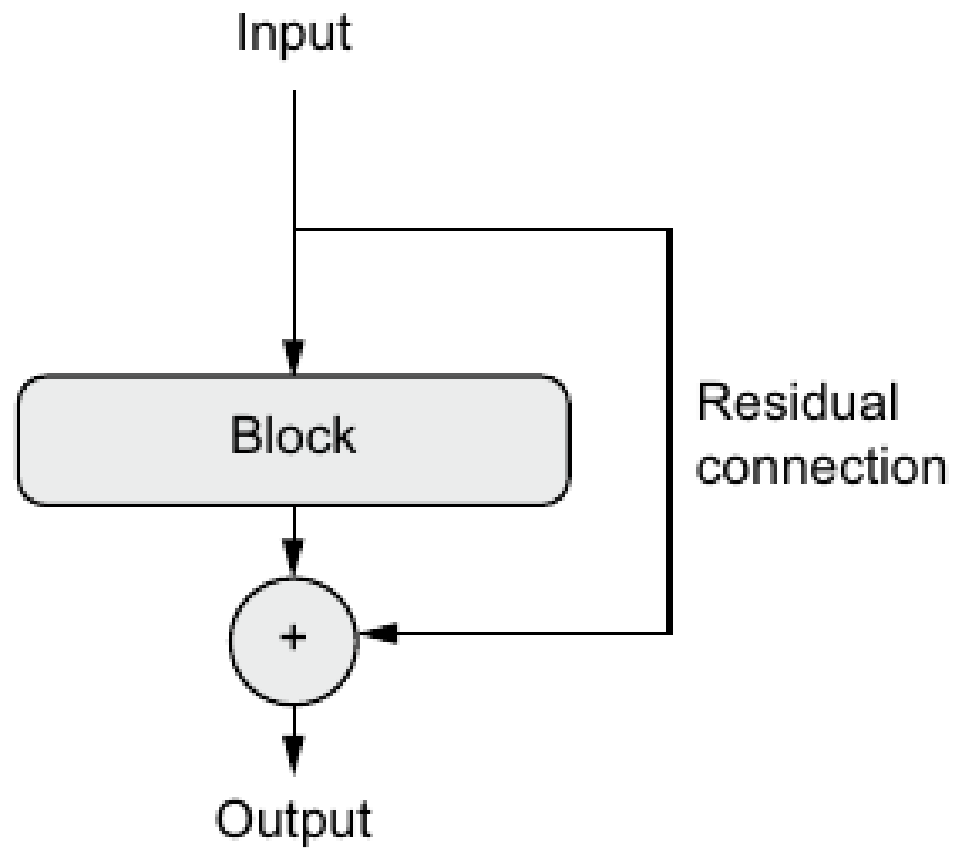


Figure 9.9 A residual connection around a processing block

Hình 9.9. Kết nối dư đi vòng qua một block xử lý.

Residual connections with shape changes Kết nối dư khi shape thay đổi

- Phép cộng residual yêu cầu hai tensor có **cùng shape**.
- Nếu số channel thay đổi, dùng **1 x 1 convolution** để chiếu residual sang shape mới.
- Nếu kích thước không gian giảm, residual branch cũng dùng **strides=2**.
- Trong Keras, thường dùng `layers.add([x, residual])`.
- Thiết kế residual giúp block sâu hơn mà vẫn train ổn định.

Batch normalization

Chuẩn hóa theo batch

- Ý tưởng chuẩn hóa:
 - x : giá trị ban đầu
 - μ (mu): giá trị trung bình của tập dữ liệu
 - σ (sigma): độ lệch chuẩn của tập dữ liệu
 - x_{norm} : giá trị sau khi chuẩn hóa
- Ý nghĩa:
 - $x - \mu$ cho biết x lệch khỏi trung bình bao nhiêu.
 - Chia cho σ để biết độ lệch đó tương đương bao nhiêu “đơn vị độ lệch chuẩn”.
- **BatchNormalization** chuẩn hóa activation trung gian của model.
- Trong training, layer dùng mean/variance của batch hiện tại.
- Trong inference, layer dùng **moving averages** đã học trong quá trình train.
- BN giúp gradient lan truyền ổn định hơn trong mạng sâu.

Using BatchNormalization well Dùng BatchNormalization đúng cách

- Thứ tự khuyến nghị trong sách:

Conv2D → BatchNormalization → Activation.

- Với BN, `Conv2D` thường đặt `use\_bias=False` vì BN đã có bước dịch chuyển.
- Activation sau BN giúp ReLU dùng mốc 0 hiệu quả hơn.
- Khi **fine-tuning**, sách khuyên để các layer BN ở trạng thái frozen.
- BN được dùng rộng rãi trong ResNet50, EfficientNet và Xception.

Depthwise separable convolutions

Tích chập tách biệt theo chiều sâu

- `SeparableConv2D` tách convolution thành hai bước.
- **Depthwise convolution**: lọc không gian riêng cho từng channel.
- **Pointwise 1 x 1 convolution**: trộn thông tin giữa các channel.
- Ví dụ tham số: $3 \times 3 \times 64 \times 64 = 36,864$ so với $3 \times 3 \times 64 + 64 \times 64 = 4,672$
- Kết quả: model nhỏ hơn, nhanh hơn và thường tổng quát hóa tốt hơn.

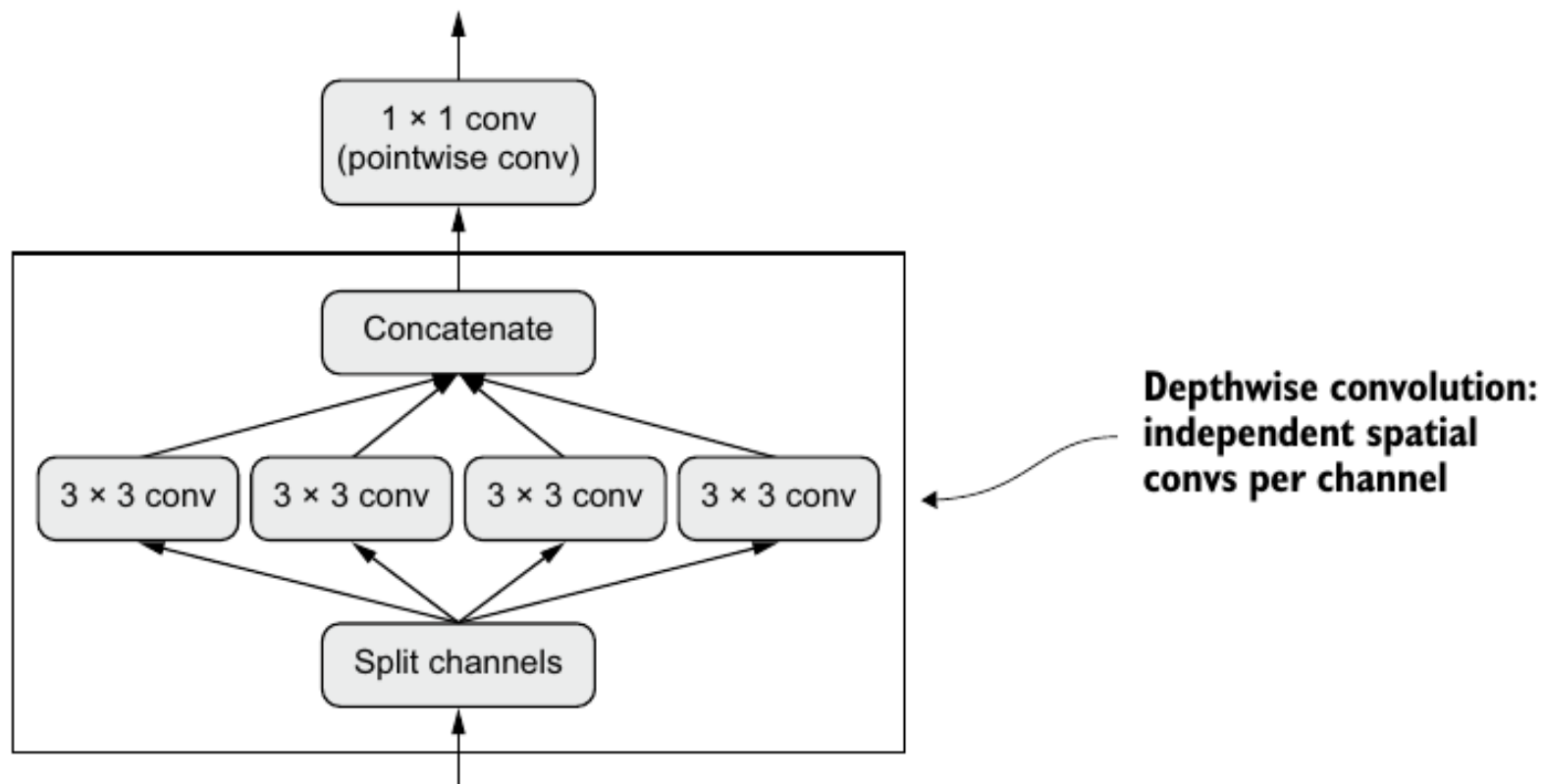


Figure 9.10 Depthwise separable convolution: a depthwise convolution followed by a pointwise convolution

Hình 9.10. Depthwise separable convolution gồm depthwise rồi pointwise convolution.

A mini Xception-like model

Một mô hình kiểu Xception thu nhỏ

- Sáp xếp kết hợp **data augmentation**, rescaling, BN, separable conv và residual blocks.
- Block lặp qua các kích thước filter: **32, 64, 128, 256, 512**.
- Layer đầu vẫn dùng Conv2D thường vì RGB channels có tương quan tự nhiên.
- Dùng **GlobalAveragePooling2D** thay cho Flatten để giảm tham số.
- Kết quả: **721,857** tham số và test accuracy khoảng **90.8%** trong ví dụ.

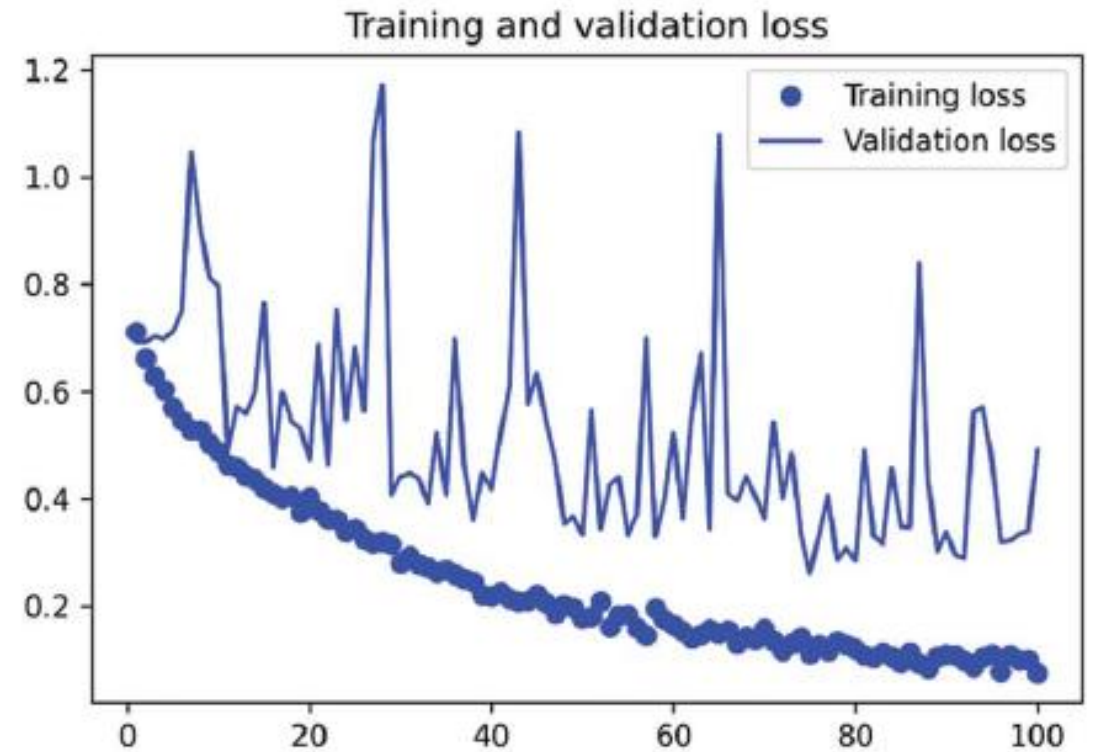
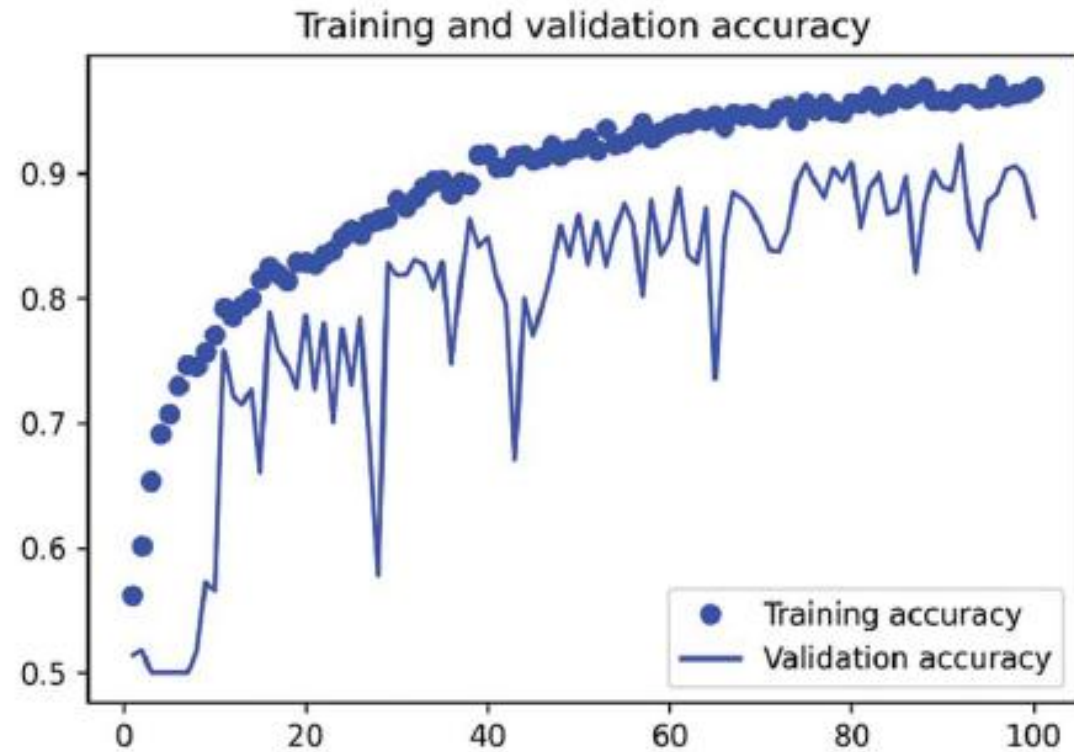


Figure 9.11 Training and validation metrics with an Xception-like architecture

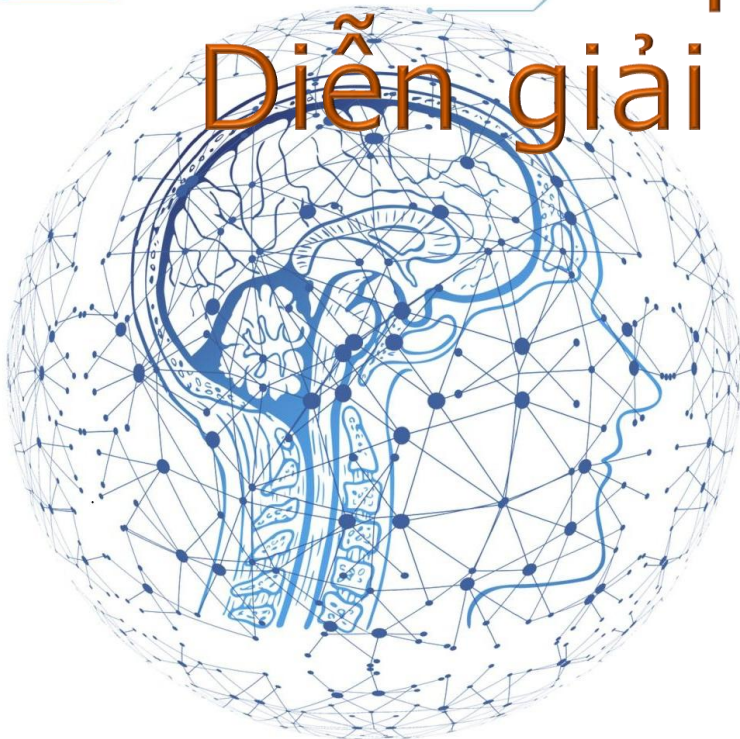
Hình 9.11. Accuracy và loss với kiến trúc kiểu Xception.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Interpreting what convnets learn

Diễn giải những gì convnet học được



Why interpret convnets? Vì sao cần diễn giải convnet?

- Convnet không hoàn toàn là **black box**: biểu diễn đã học có thể được quan sát.
- Diễn giải giúp debug lỗi, hiểu model và tăng độ tin cậy trong ứng dụng nhạy cảm.
- Ba kỹ thuật trong sách: **activation visualization, filter visualization, class activation heatmap**.
- Mỗi kỹ thuật trả lời một câu hỏi khác nhau về model.
- Mục tiêu không phải chứng minh tuyệt đối, mà là tạo **bằng chứng trực quan** về hành vi model.

Intermediate activations

Activation trung gian

- Tạo model mới trả về output của nhiều layer `Conv2D` và `MaxPooling2D`.
- Một activation có shape dạng `(batch, height, width, channels)`.
- Mỗi channel là một **feature map** 2D.
- Layer đầu thường giống edge/texture detectors và còn giữ nhiều thông tin ảnh.
- Layer sâu hơn trừu tượng hơn, thưa hơn và khó diễn giải bằng mắt hơn.

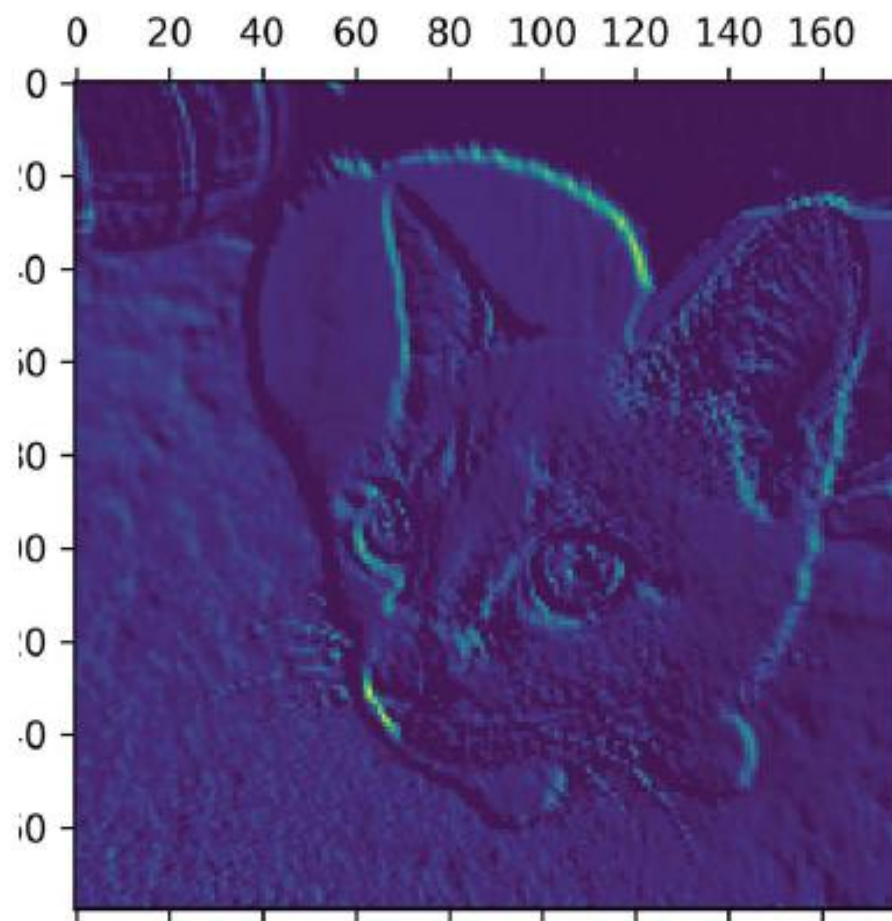


Figure 9.13 Fifth channel of the activation of the first layer on the test cat picture

Hình 9.13. Channel thứ năm của activation ở layer đầu tiên.

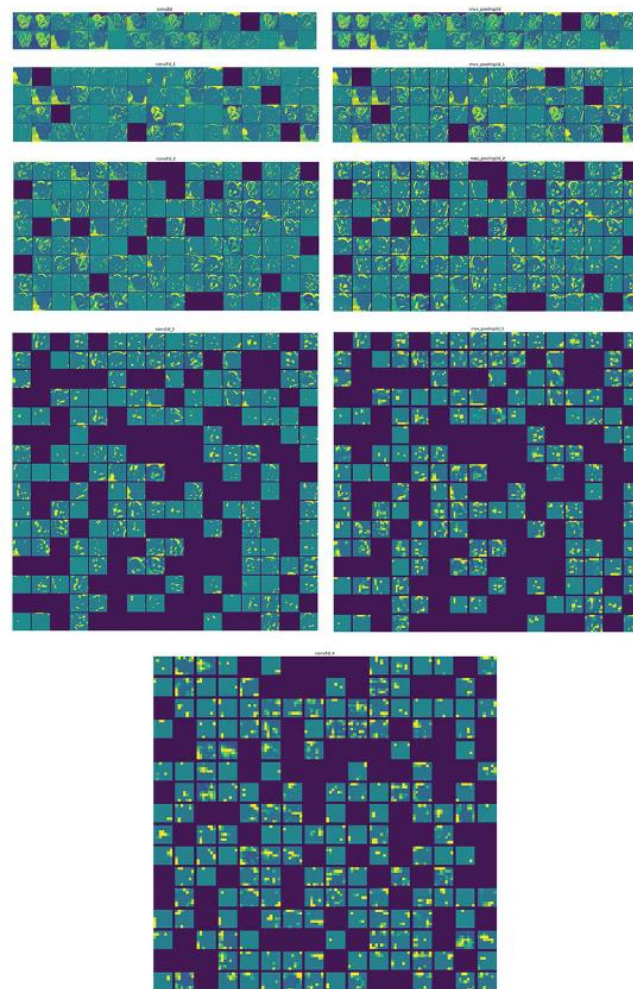


Figure 9.14 Every channel of every layer activation on the test cat picture

Hình 9.14. Tất cả channel của mọi layer activation trên ảnh thử.

Visualizing filters

Trực quan hóa filter

- Ta tìm ảnh đầu vào làm một filter kích hoạt mạnh nhất.
- Dùng **gradient ascent** trên input image, không phải trên weight.
- Công thức cập nhật:

$$x \leftarrow x + \eta \nabla_x L$$

- Loss thường là mean activation của filter cần quan sát.
- Filter sâu hơn thường học các pattern phức tạp như texture, motif hoặc bộ phận đối tượng.

- Trong đó:

- x : biến đang được cập nhật, ví dụ ảnh đầu vào, tham số, hoặc dữ liệu.
- η (eta): tốc độ học, hay bước cập nhật.
- $\nabla_x L$: gradient của hàm mất mát L theo x . (∇ : nabla).

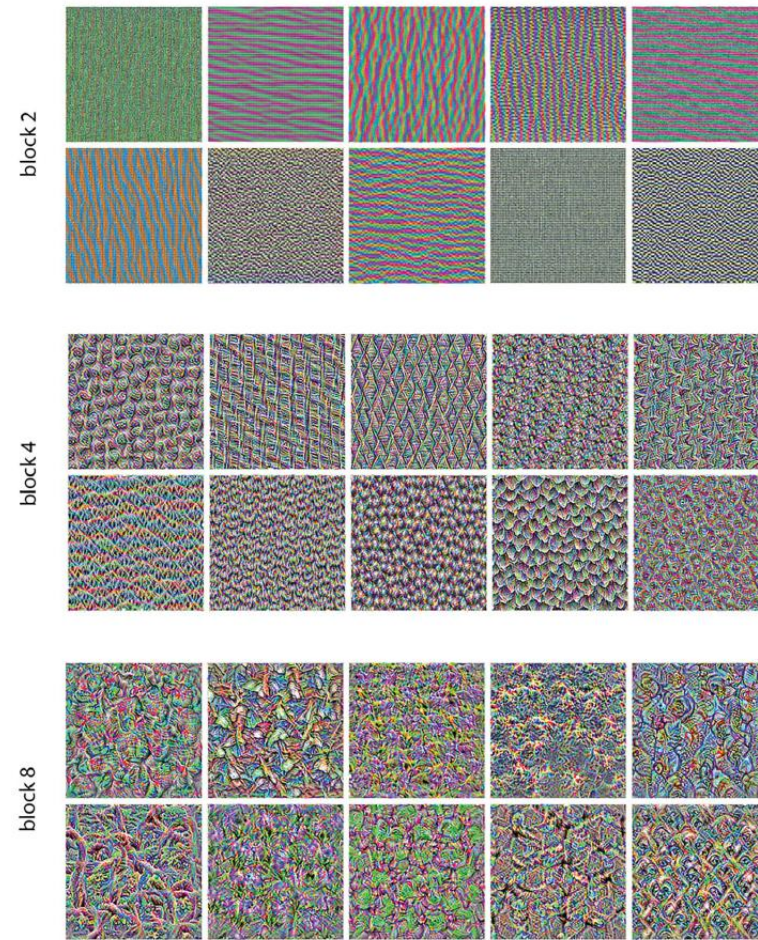


Figure 9.17 Some filter patterns for layers `block2_sepconv1`, `block4_sepconv1`, and `block8_sepconv1`

Hình 9.17. Một số pattern filter ở các block khác nhau.

Class activation heatmaps

Bản đồ nhiệt kích hoạt lớp

- **Grad-CAM** cho biết vùng ảnh nào quan trọng cho một class cụ thể.
- Lấy gradient của score class đối với output của layer convolution cuối.
- Tính trọng số channel: $\alpha_k = \text{mean}_{i,j}(\partial y_c / \partial A_{ij}^k)$
- Heatmap đơn giản: $H = \frac{1}{C} \sum_k \alpha_k A^k$
- Heatmap được chồng lên ảnh gốc để diễn giải quyết định của model.

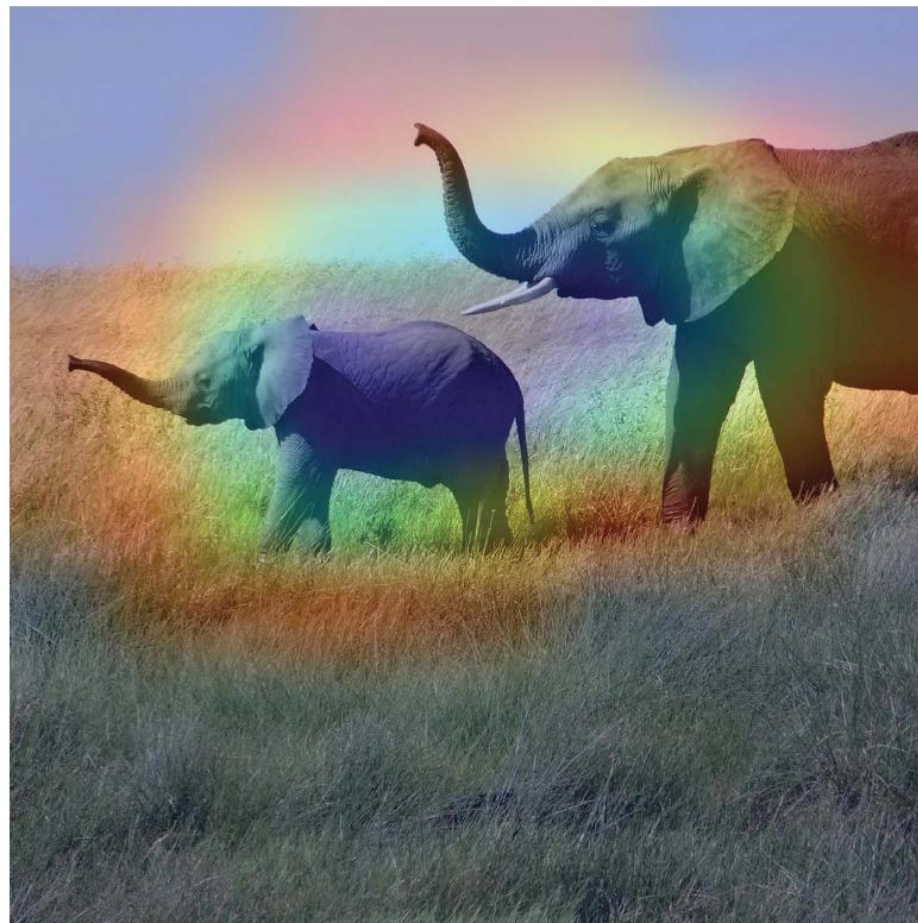


Figure 9.20 African elephant class activation heatmap over the test picture

Hình 9.20. Heatmap class 'African elephant' chồng lên ảnh thử.

Chapter summary

Tổng kết chương

- Computer vision gồm nhiều tác vụ: **classification, segmentation, detection** và hơn nữa.
- Segmentation dự đoán **class cho từng pixel**, nên cần giữ thông tin không gian tốt.
- Kiến trúc convnet hiện đại dùng **residual connections, batch normalization, separable convolutions**.
- Thiết kế tốt dựa trên **modularity, hierarchy, reuse** và kiểm chứng bằng ablation study.
- Interpretability giúp quan sát activation, filter và vùng ảnh ảnh hưởng đến dự đoán.

